

Step 4 — Test Gate + Rollback — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — The suite, the gate, and what "last good version" really means](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The suite, the gate, and what "last good version" really means

Basic Version

Three separate pieces, built in order: a test suite (a fixed set of topics with rules for what a "good" result looks like), a gate (won't let a new version go "live" unless it passes the suite), and a rollback (goes back to a version that's known to have passed).

The suite comes first because the gate and rollback are both meaningless without it — you can't refuse a bad version, or find the last good one, if nothing defines "good" yet.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

A "version" here is a snapshot of whatever's tunable — most simply, the Writer/Reviewer prompts and any key settings (temperature, `MAX_ROUNDS`). Store each version with an ID (a timestamp or short hash works fine), the actual prompt/setting values, and a fixed test suite result once it's been run: a list of fixed topics run through the graph, each scored against simple pass/fail rules (does the report mention key facts from the notes? is it non-empty? did it get approved by the Reviewer within the round limit?) or an LLM-judge score.

The gate is a function: given a candidate version, run the suite against it, and only mark it "passed" (and eligible to be "live") if every rule passes (or the score clears a threshold). Store every version's suite result — passed or failed — not just the currently-live one; that history is exactly what rollback needs.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Read the README's rollback requirement literally: "goes back to the last version that actually passed, not just 'whatever came before.'" That sentence is warning against a specific, easy-to-write bug: a rollback that just does `versions[-2]` (the previous version in the list) without checking whether *that* version passed either. Picture this real sequence: v1 passes, v2 fails (a bad prompt change), v3 also fails (someone tried to fix v2 and made it worse), and now you want to roll back. "Whatever came before" (v3's predecessor) is v2 — which never passed either. The correct rollback has to walk *backward* through version history until it finds one whose stored suite result was actually "passed", skipping over any failed versions in between, however many there are.

There's a second, related correctness issue: what does the gate do with a version that's never been tested at all — say, one added directly to storage without ever running through the gate? It should never be treated as "passed" by default. Absence of a failure is not the same as a recorded pass, and a rollback (or a "what's currently live" check) that doesn't distinguish "explicitly passed" from "never tested" can accidentally promote something that was never actually verified.

The extra pieces:

- Store `status` on every version as one of exactly "passed", "failed", or "untested" — never let "not failed" get treated as equivalent to "passed."
- Rollback scans version history **backward from the most recent**, returning the first one with `status == "passed"` — not simply "the previous version," and not stopping at the first "failed" one it happens to hit.
- If no version in the entire history has ever passed, rollback should fail loudly (raise, don't silently do nothing or silently pick something untested) — there's genuinely nothing safe to roll back to yet.

Sketch the backward-scanning rollback yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both assume "the version before this one" is a safe rollback target. Advanced treats that assumption as false by default — it only trusts a version's *recorded, explicit* pass result, walks back through as much history as it takes to find one, and refuses to guess if none exists.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
eval_suite/tasks.py:
    a fixed list of test topics, each with a simple pass/fail rule
    (e.g. "the report isn't empty", "the report mentions at least one number")

eval_suite/run.py:
    def run_suite(version) -> bool:
        run each test topic through the graph using this version's settings
        check each result against its rule
        return True only if every task passes

deploy_gate.py:
    def try_deploy(version):
        if run_suite(version): mark version as "passed" and make it live
        else: mark version as "failed", refuse to make it live

rollback.py:
    def rollback():
        look through saved versions, newest to oldest
        return the first one marked "passed"
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
eval_suite/tasks.py:
    EVAL_TASKS = [
        {"topic": "electric bikes", "rule": lambda report: len(report) > 100},
        {"topic": "remote work", "rule": lambda report: "productivity" in report.
        ...
    ]

eval_suite/run.py:
    def run_suite(prompt_overrides: dict) -> dict:
        # returns {"passed": bool, "results": [per-task detail]}
        results = []
        for task in EVAL_TASKS:
            report = run_graph with prompt_overrides applied, on task["topic"]
            results.append({"topic": task["topic"], "passed": task["rule"](report)
        return {"passed": all(r["passed"] for r in results), "results": results}

versioning.py:
    class Version(TypedDict):
        id: str
        prompt_overrides: dict
        status: str # "passed" | "failed" | "untested"
        created_at: datetime

deploy_gate.py:
    def try_deploy(version: Version) -> bool:
        suite_result = run_suite(version["prompt_overrides"])
        version["status"] = "passed" if suite_result["passed"] else "failed"
        save_version(version) # always saved, passed or failed
        if version["status"] == "passed":
            set_live_version(version["id"])
        return version["status"] == "passed"

rollback.py:
    def rollback() -> Version:
        for version in all_versions_newest_first():
            if version["status"] == "passed":
                set_live_version(version["id"])
                return version
        raise NoPassingVersionError("no version has ever passed the suite")
```

Test with a version made worse on purpose (e.g. a prompt that omits key instructions) — confirm the gate refuses it, and confirm a rollback afterward correctly restores the last version that actually passed. Write it yourself, then compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's almost the version history walk — fill in the missing scan yourself:

```
class NoPassingVersionError(Exception):
    pass

def rollback() -> Version:
    versions = all_versions_newest_first() # includes "passed", "failed", AND "u

    # your turn: walk `versions` in order (already newest-first), and
    # return the first one whose status is exactly "passed" — skip over
    # any "failed" or "untested" ones, however many there are in a row.
    # If none is found after checking every version, raise
    # NoPassingVersionError("no version has ever passed the suite").
    ...
```

Fill in the scan, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same 3-piece system (suite, gate, rollback) at 3 completeness levels — Basic and Intermediate's rollback logic is described as "find the first passed one," but neither is explicit about correctly skipping over *multiple consecutive* failed versions, or about refusing to guess when nothing has ever passed. Advanced makes both of those an explicit, tested part of the scan.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)